

HYPERSHIELD: Protecting the Hypercube MPC-in-the-Head Framework Against Differential Probing Adversaries without Masking

Linda Pirker ^{1,*}, Quinten Norga ², Suparna Kundu ², Anindya Ganguly ³, Barry van Leeuwen ², Angshuman Karmakar ³ and Ingrid Verbauwhede ²

¹ Graz University of Technology, Graz, Austria

linda.pirker@student.tugraz.at

² COSIC, KU Leuven, Leuven, Belgium

{quinten.norga,suparna.kundu,barry.vanleeuwen,ingrid.verbauwhede}@esat.kuleuven.be

³ Indian Institute of Technology Kanpur, Kanpur, India

{anindyag,angshuman}@cse.iitk.ac.in

Abstract. Post-quantum secure digital signatures based on the MPC-in-the-Head (MPCitH) paradigm, a zero-knowledge (ZK) proof-based construction, are becoming increasingly popular due to their small public key size. However, the development of techniques for protecting MPCitH-based schemes against side-channel attacks remains slow, despite them being critical for real-world deployment.

In this work, we adapt the Hypercube-MPCitH framework exploiting its native use of additive secret sharing to enable inherent protection against first- and high-order differential power analysis (DPA). We first perform a sensitivity analysis of the Hypercube Syndrome Decoding in the Head (SDitH) digital signature scheme with respect to both simple and differential power analysis. Based on this insight, we then propose a tweak to the signature scheme to increase its *inherent* resistance against DPAs *by design*, eliminating the need to explicitly mask large parts of the signing procedure. More specifically, this is achieved through the novel $(k + 1)$ -Hypercube ZK Protocol: the proposed tweak increases the number of hidden shares an adversary must probe to recover the secret key from one to $k + 1$, thus achieving *inherent* masking order k . Typically, increasing the amount of hidden shares results in a degradation of soundness in the zero-knowledge proof and as a result increases the signature size to a point where the scheme becomes of limited practical interest. To address this, we propose a technique to select the *hidden* shares in a more structured and optimal fashion, by exploiting the GGM tree structure in the Hypercube-MPCitH framework. As a result, the amount of revealed seeds is reduced, thus resulting in a smaller signature size even compared to the original hypercube protocol.

Finally, we implement and benchmark the proposed Hypercube-SDitH signature scheme, comparing it against the cost of traditional masking. For 3rd-order protection, our tweaked signature scheme only incurs a 35 – 50% overhead in computational cost, compared to an estimated overhead of 300% for a fully masked implementation, while the overhead in signature size stays relatively low (52%). Overall, we demonstrate that the proposed $(k + 1)$ -Hypercube ZK Protocol can be used to construct efficient, DPA-resistant MPCitH-based digital signatures.

Keywords: Side-Channel Analysis · MPC-in-the-Head · Post-Quantum Cryptography · Digital Signatures.

*Part of this work was completed while the author was at COSIC, KU Leuven.

1 Introduction

The constant progress in the development of large-scale quantum computers poses a serious threat to the security of widely used public-key cryptosystems, such as RSA [RSA78] and elliptic curve systems [Kob87, Mil85]. Acknowledging this urgency, NIST launched a competition to standardize quantum-resistant public-key cryptography, including digital signature schemes [NIS, AAC⁺22]. The first standardization effort concluded with the selection (and standardization) of multiple signature schemes: ML-DSA [DLL⁺18], FN-DSA [FHK⁺18] and SLH-DSA [BHK⁺19]. To increase diversity in standardized schemes, NIST launched an additional call for digital signatures schemes, explicitly focused on standardizing schemes based on different mathematical problems [NIS22].

Signature schemes based on the MPC-in-the-Head (MPCitH) framework are a promising class of candidates in this ongoing competition [NIS24], including SDitH [FJR22], MQOM [BFR24], PERK [BBD⁺24] or RYDE [BCDF⁺25]. In MPCitH-based digital signature schemes, a zero-knowledge proof is constructed using secret sharing techniques from secure multi-party computation (MPC) [IKOS09]. More specifically, the prover uses a zero-knowledge proof to prove knowledge of a secret key to the verifier. This proof is split into multiple partial proofs on (a typically large amount of) shares of a (secret) key-derived witness and the execution of those partial proofs is emulated locally (“in-the-Head”) by the signer. Eventually, a number of shares lower than the privacy threshold of the used secret sharing scheme (e.g. additive sharing) is revealed in the signature, allowing the verifier to recompute and verify parts of the proof without being able to recover the secret key. After the initial proposal of the MPCitH framework, several optimized versions like Threshold-Computation-in-the-Head [FR25], Hypercube-MPCitH [AMGH⁺23] or VOLE-in-the-Head [BBdSG⁺23] have been developed, which reduce the amount of party computations that need to be performed or the signature size. Hypercube-MPCitH, used by SDitH [MFG⁺23], is particularly interesting due to the small signature sizes and its simple and scalable structure, utilizing the generally cheap and easy-to-implement additive secret sharing.

Power analysis attacks, most famously Differential Power Analysis (DPA) [KJJ99], are a type of passive Side-Channel Attack (SCA) which poses a serious threat to cryptographic implementations, especially on embedded devices [LBC17, KGB⁺18, BDT⁺23]. These power-based SCAs enable attackers to recover secret information, by measuring and exploiting statistical dependencies of the processed data with the device’s power consumption. Masking is a well-studied countermeasure to construct d -probing secure implementations, resistant against d -order DPAs [ISW03]. This countermeasure transforms the cryptographic functions to operate on $d + 1$ randomized shares of sensitive data and thereby breaks dependencies between power consumption and secret data.

However, the application of masking to MPCitH-based digital signature schemes is expensive due to the nature of its zero-knowledge proof framework. More specifically, to achieve d -order protection, each of the n party computations of a share must be masked, which essentially amounts to further creating d shares of each party’s share of the secret, thus $n \cdot (d + 1)$ in total [ABE⁺21]. As the cost of masking Hypercube-MPCitH schemes is implicitly high, the question arises *whether the underlying Hypercube-MPCitH framework and its secret sharing can be tweaked to achieve implicit and efficient side-channel protection?*

Related Works Side-channel protection of MPCitH signature schemes, especially recent NIST candidates, is a almost unexplored topic. Picnic [CDG⁺17], which was submitted to the first NIST standardization competition, uses secret sharing as an integral part of its structure. Yet it has been shown to be vulnerable to DPA [GSE21]. In [SBWE20], the authors show that certain modifications to the scheme allow to utilize some inherent

protection provided by the secret sharing at a loss of soundness of the zero-knowledge proof, which can be accounted for by adjusting the protocol parameters. However, the proposed strategy is not directly applicable to protect Hypercube-MPCitH signature schemes as Picnic relies on decompositions of LowMC [ARS⁺15] block ciphers.

More recent work by Feneuil et al. [FRWJ25] is directed towards the masking-friendliness of the related Threshold-Computation-in-the-Head (TCitH) framework [FR25], reducing the number of calls to masked symmetric primitives. Additionally, the authors show that the tweak initially proposed for Picnic can also be applied to the TCitH framework, however leading to a large overhead in signature which limits practical applicability.

Our Contributions Previous work on tweaking other variants of MPCitH signature scheme to achieve implicit side-channel protection has yielded mixed results, mainly due to relatively large overheads in signature size and computational cost. In this work, we present a tweaked Hypercube-MPCitH framework that achieves inherent side-channel resistance without relying on the masking countermeasure, at a lower cost than other variants of the framework. More specifically:

1. We provide a detailed analysis of side-channel attack paths in the SDitH signature scheme, which is constructed using the Hypercube-MPCitH framework. We identify operations and subroutines that are vulnerable to SPAs and DPAs. (Section 3)
2. We propose the novel $(k + 1)$ -Hypercube ZK Protocol: a tweaked hypercube-based zero-knowledge protocol that achieves inherent high-order side-channel security against DPAs, *by design*. This is achieved by introducing slack in the open parties, thus revealing fewer shares than the reconstruction threshold, increasing the number of shares an adversary needs to probe to recover the secret. Further, we propose an optimized approach for selecting hidden shares in GGM trees, further reducing the signature size. We also provide a proof of completeness, soundness and the zero-knowledge property of our tweaked protocol. (Section 4)
3. We demonstrate how the proposed tweak can be used to construct a hardened Hypercube-SDitH signature scheme that achieves inherent k -order probing security for a significant part of the signing procedure.¹ Finally, we analyse its overhead compared to a fully masked implementation and the recently proposed masking-friendly TCitH scheme, for each of our proposed parameter sets, which provide a trade-off between speed and signature size. (Section 5)

2 Preliminaries

We denote by $\llbracket x \rrbracket = (\llbracket x \rrbracket_1, \dots, \llbracket x \rrbracket_n)$ the sharing of a value x . The k -th share is denoted $\llbracket x \rrbracket_k$. To prevent confusion between the usual notation of N as the number of parties or shares in literature and the hypercube parameter N for the side of the hypercube, we always refer to the number of parties with n .

2.1 Zero-Knowledge Proofs

Zero-knowledge proofs (ZKP) are interactive protocols in which a prover $\mathcal{P}$ proves a statement to a verifier $\mathcal{V}$, typically the knowledge of a witness for some relation. The messages exchanged during the protocol are called transcripts of the execution. A ZKP must fulfill the following properties: (i) *Completeness*, an honest $\mathcal{P}$ will always succeed in convincing $\mathcal{V}$; (ii) *Soundness*, no cheating $\tilde{\mathcal{P}}$ can succeed in convincing $\mathcal{V}$, except with

¹Our implementation will be made publicly available after publication.

a small probability ε , also known as the soundness error; and (iii) *Zero-Knowledge*, the transcript of an execution does not reveal any information about the witness.

2.2 (Additive) Secret Sharing

Secret sharing is a technique from multi-party computation (MPC) to distribute a secret (split into shares) between multiple parties, ensuring only a *qualified* subset can reconstruct the secret. Additive sharings are n -out-of- n secret sharings, which means that all n shares have to be known to reconstruct the secret. An additive secret sharing $\llbracket x \rrbracket_{k \in [1, n]}$ of the variable x between n parties is defined such that $x = \sum_{k=1}^n \llbracket x \rrbracket_k$. In practice, they are usually constructed by choosing the first $n - 1$ shares at random and choosing the last share, also known as *auxiliary share*, such that $\llbracket x \rrbracket_n = x - \sum_{k=1}^{n-1} \llbracket x \rrbracket_k$.

2.3 MPC-in-the-Head Framework

The MPCitH paradigm is based on the work of Ishai et al. [IKOS09], who first described a general construction of a zero-knowledge proof using a MPC protocol combined with a commitment protocol. The framework allows a prover to prove the knowledge of a secret x to a verifier by executing an MPC protocol among n parties. The parties jointly evaluate a proof Π that either outputs ACCEPT for a valid secret x or REJECT for an invalid secret. In more detail, the following steps are performed:

- The prover commits to n shares $\llbracket x \rrbracket$ of the secret x . She simulates MPC locally (“*in-the-Head*”) to compute proofs and commits to the views of the parties. Further, she sends the n output shares $\llbracket \Pi(x) \rrbracket$ of the parties to the verifier.
- The verifier chooses a subset of $t \leq n - 1$ parties and asks the prover to reveal the view of the parties. The zero-knowledge property is preserved, as up to $n - 1$ views do not reveal any information about the secret x .
- After receiving the views of the revealed parties, the verifier checks the consistency of the commitments by recomputing the t party computations. Additionally it verifies that the recombined output shares $\llbracket \Pi(x) \rrbracket$ are evaluated to ACCEPT.

A verifier without knowledge of the secret x can either generate a signature that evaluates to $\Pi(x) = \text{ACCEPT}$ with a false positive probability of p or cheat by corrupting one party computation, which will be undetected with a probability of $\frac{n-t}{n}$. This results in a total soundness error of $p + (1-p)\frac{n-t}{n}$ of the zero-knowledge protocol. To increase the soundness of the proof, τ parallel iterations of the MPC protocol are performed.

2.4 Hypercube-MPCitH

The hypercube optimization can be applied to any MPCitH signature scheme that uses additive secret sharing between n parties, given that n can be expressed as $n = N^D$. The framework follows a geometrical approach arranging the N^D shares (also called leaf shares) in a hypercube of side N and dimension D to translate the original sharing between N^D parties to D sharings between N main parties. The main party shares for each of the D dimensions are constructed by grouping the original leaf shares into N slices of the hypercube along the corresponding dimension. The additive secret sharing scheme allows to accumulate the shares within each slice, yielding D sets of sharings between N parties encoding the same original secret.

Within the hypercube, the leaf shares are indexed either using their absolute index s_i with $i \in [1, N^D]$ or by indexing them in the respective dimension $\mu \in [1, D]$ using the index $s_{\mathbf{i}}$ with $\mathbf{i} = (i_1, \dots, i_D)$ and $i_\mu \in [1, N]$. The main shares are always indexed using the dimension μ and the index ν within this dimension $s_{\mu, \nu}$ with $\mu \in [1, D]$ and $\nu \in [1, N]$.

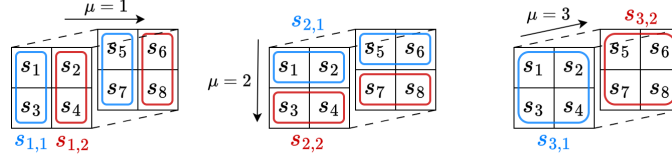


Figure 1: Construction of main party shares on hypercube of $N = 2$ and $D = 3$

Figure 1 shows how a hypercube of side $N = 2$ and dimension $D = 3$ is used to construct 3 sets of 2 main shares from $2^3 = 8$ original shares/parties. The main shares for dimension $\mu = 1$ are $s_{1,1}$ and $s_{1,2}$ (left); along dimension $\mu = 2$ they are $s_{2,1}$ and $s_{2,2}$ (middle); dimension $\mu = 3$ yields $s_{3,1}$ and $s_{3,2}$ (right).

As all D main party sharings encode the same plain value, further optimization is possible. Performing a single computation using the plain value s allows to omit one party computation per dimension. Instead, this last share can be deduced from the difference of the first $N - 1$ parties computations and the plain computation, reducing the amount of required party computations from N^D to $1 + (N - 1) \cdot D$. For the example in Figure 1, the party computations are reduced from $2^3 = 8$ to $1 + (2 - 1) \cdot 3 = 4$.

2.5 Power Analysis Attacks

Power analysis attacks are categorized based on attacker capabilities: for each one of the classes, different countermeasures are suitable.

DPA are characterized by the use of a large number of power traces to detect statistical dependencies on secret data [KJJ99]. Since power traces from multiple executions of the operation are required to mount a DPA, it can only be used to recover long-term secrets. Ephemeral secrets, which are randomly generated for each execution of the operation, cannot be probed using DPA. Protection against DPA is studied extensively and includes masking [CJRR99] as the most prominent countermeasure.

Contrary to DPA-type attacks, single-trace-type attacks can be performed using only a single power trace allowing an attacker to probe ephemeral secrets as well as long-term secrets. Classic Simple Power Analysis (SPA) as first proposed in [KJJ99] as well as template attacks [CRR02] or more recent deep-learning based attacks [CDP17] fall into this class of attacks. Masking is not considered to be an efficient countermeasure to protect against single-trace-type attacks and instead additional countermeasures like shuffling [HOM06] have to be applied [BGS15, PP19, AEVR23].

3 Sensitivity Analysis of Hypercube-SDitH

In the following section, we briefly introduce the Hypercube-SDitH signature scheme [FJR22, MFG⁺23], as submitted to the first round of the NIST call for *Additional Digital Signature Schemes* [NIS22]. Subsequently, we perform an analysis of the different power analysis attack paths possible in its signing procedure, based on the SDitH specification and reference implementation².

3.1 The SDitH Signature Scheme

Hypercube-SDitH uses the syndrome decoding (SD) problem and builds a signature scheme based on the MPCitH framework. The Fiat-Shamir heuristic is used to make the

²<https://github.com/sdith/sdith>

zero-knowledge proof non-interactive.

Construction of the Zero-Knowledge Proof The prover proves knowledge of a valid solution x to a public SD instance (H, y) such that $y = Hx$ and $\text{wt}(x) = w$ where $\text{wt}(x)$ is the Hamming weight i.e. the number of bits set in the variable x . To build a proof, three polynomials $Q(X)$, $S(X)$ and $P(X)$ are derived from the witness x and a public polynomial $F(X)$ is used such that $S(X) \cdot Q(X) = F(X) \cdot P(X)$. Verifying a valid solution amounts to verifying the polynomial $F(r) \cdot P(r) - S(r) \cdot Q(r) = 0$ for a series of challenge points r . This is done in a MPC setting, using a valid set of Beaver triples a, b, c such that $a \cdot b = c$, following the approach described in [BN20]:

$$\llbracket \alpha \rrbracket_i = \varepsilon \cdot \llbracket Q(r) \rrbracket_i + \llbracket a \rrbracket_i \quad \llbracket \beta \rrbracket_i = \llbracket S(r) \rrbracket_i + \llbracket b \rrbracket_i \quad (1)$$

$$\llbracket v \rrbracket_i = \varepsilon \cdot \llbracket F(r) \cdot P(r) \rrbracket_i - \llbracket c \rrbracket_i + \alpha \cdot \llbracket b \rrbracket_i + \beta \cdot \llbracket a \rrbracket_i - \alpha \cdot \beta \quad (2)$$

Equation 1 shows the shares of the broadcast values, which are used to jointly compute the plain broadcast values $\alpha = \sum_{i=1}^n \llbracket \alpha \rrbracket_i$ and $\beta = \sum_{i=1}^n \llbracket \beta \rrbracket_i$ necessary for the computation of the parties output share $\llbracket v \rrbracket_i$ in Equation 2. If the shares recombine to $v = \sum_{i=1}^n \llbracket v \rrbracket_i = 0$, the proof is accepted. To reach the required level of soundness, the proof is repeated τ times.

Signature Generation Below is a high-level description of the signature generation algorithm for the hypercube variant of the SDitH signature scheme. For the complete specification, we refer to its first round specification document [MFG⁺23].

- For each of the τ repetitions, $n = N^D$ leaf seeds are expanded from a root seed using a GGM tree. The first $n - 1$ leaf shares are directly expanded from the $n - 1$ leaf seeds while the n -th leaf share is constructed additionally using the accumulated $n - 1$ leaf shares as auxiliary value. The leaf shares are hashed to create the commitments.
- The $N - 1$ main input shares of the D dimensions are built by accumulating the corresponding hypercube slices, while the main share containing the n -th leaf share is omitted (and thus requires the auxiliary value).
- The MPC challenges (r, ε) for the τ parallel repetitions are expanded from the hashed commitments to the shares and are used to perform the plain broadcast computation as well as the party computations.
- The view challenges I^* for the τ parallel repetitions are expanded from the hashed plain broadcast values and $N - 1$ main parties broadcast values for each dimension. It corresponds to the index of the hidden leaf share $I^* = i^* \in [1, N^D]$ or equivalently indexed per dimension μ in the hypercube $I^* = \mathbf{i}^* = (i_1^*, \dots, i_D^*)$ where $i_\mu^* \in [1, N]$. All other leaf parties are included in the signature. The use of the GGM tree allows to do that efficiently, by revealing D sibling seeds.

3.2 Sensitivity Analysis

The signing algorithm takes the public parameters $(H', y,)$ and the key-derived witness (Q, S, P) as input and returns the signature $\sigma = (\text{salt}, h_2, \text{seed}_{\text{sib}}, \text{aux}, (\alpha, \beta), \text{com}_{i \in I^*})$. We present the results of our sensitivity analysis of the signature generation in Figure 2. For simplicity, parallel repetitions ($\tau > 1$) are omitted. The seed expansion using the GGM tree produces N^D leaf seeds which are further expanded using an XOF (extendable output function) into N^D sets of $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket, \llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$ (of which $k + 1$ are secret) and an auxiliary value aux (which may also be secret) to complete the sharing. Those leaf

shares are hashed to create N^D commitments to the leaf shares, which are further hashed and expanded to the MPC challenges (r, ε) on which the plain broadcast computation Π_{plain} is performed using the secret plain (Q, S, P) . The main shares are constructed for each dimension and the MPC computations Π are performed using them. Finally, the output is hashed to expand the view challenge I^* , which decides which exact $(k + 1)$ leaf shares are kept hidden i.e. remain secret.

In our analysis, we distinguish between i) secret variables such as secret-key derived witness-polynomials (Q, S, P) , which are vulnerable to DPAs, ii) ephemeral secrets such as the Beaver triples (a, b, c) , which are vulnerable to SPAs and iii) public values (see Section 2.5).

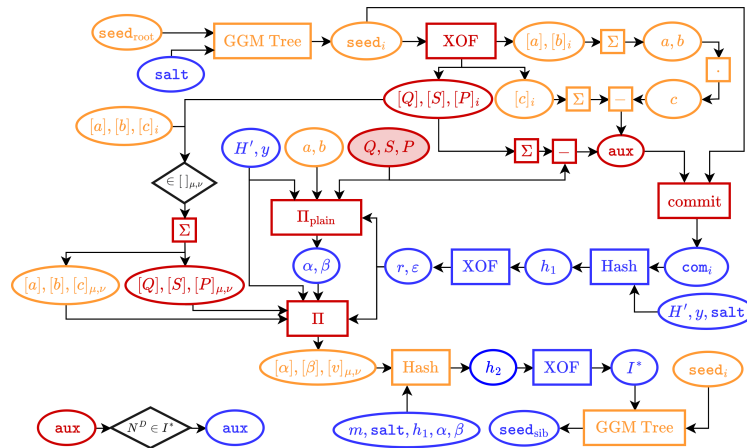


Figure 2: Sensitivity analysis of the Hypercube-SDitH signing algorithm. Red filled: long-term secrets. Red: vulnerable to DPA. Orange: vulnerable to SPA. Blue: public (not vulnerable).

DPA Susceptibility Recovering the polynomial S will allow an attacker to recover the secret SD solution as shown in the attack on SDitH demonstrated in [GAG⁺25]. The recovery of polynomials P, Q might not allow full recovery of the secret key, but at least discloses information that decreases the key space. It further extends to shares of the witness-polynomials ($[Q], [S], [P]$) and the auxiliary value $\mathbf{aux}$, which is constructed by accumulating those shares and plain values.

All but one of those shares are made public in the signature and therefore can be used by an attacker to recover the long-term secret, only using a single probe. More specifically, by using the revealed party shares (all-but-one) and guessing the encoded long-term secret, the power consumption (and thus value) of the hidden witness-polynomial share can be modeled. For example, consider a 3-party additive sharing of a secret variable x such that $x = [x]_1 + [x]_2 + [x]_3$. If shares $[x]_1$ and $[x]_2$ are public, an attacker could use DPA to verify a guess x' of the plain secret x by modeling the power consumption of the hidden share as $[x]_3 = [x]_1 + [x]_2 + x'$.

Proposals for countermeasures against this type of attack include the application of masking to each party’s share [ABE⁺21] or the introduction of a slack in revealed parties i.e. the use of an increased number of hidden parties to design a side-channel secure ZK protocol [SBWE20]. We rely on the latter to build an inherently DPA-resistant Hypercube ZK Protocol in Section 4.

Single-Trace-Type Attack Susceptibility Ephemeral secrets are generally vulnerable to single-trace attacks. Probing any of the Beaver triples (a, b) allows an attacker to recover the evaluation of either (Q, S) in a public point, and collecting $\deg(Q)$ or $\deg(S)$ evaluations is enough to reconstruct the corresponding polynomial. It also extends to the shares of the Beaver triples $([\alpha], [\beta], [v])$, however with an increased attack complexity. Both the variable $[\alpha]$ as well as the variable $[a]$ would have to be probed to recover the hidden share $[Q]$ in a single-trace attack. The same applies for $[\beta]$ and $[b]$ for $[S]$. The recovery of $[P]$ would even require $[v]$, $[a]$, $[b]$ and $[c]$ to be probed. Further the root, intermediate and leaf seeds fall into this category, as probing them can potentially enable an attacker to recompute secret hidden shares. Single-trace-type attacks are inherently more challenging and require complementary countermeasures such as shuffling or parallelism and are considered out-of-scope for this work.

3.2.1 GGM Seed Tree Expansion

Although a probing attack on the root or intermediate seeds in the GGM tree poses a serious threat and thus requires countermeasures, such an attack can not be performed on the SDitH scheme using accumulated probes as is the case for DPA-type attacks. Hence, given that one-time expansion of the root seed can be guaranteed (by correctly sampling it from a proper source of randomness for each signature generation), any attack targeting the seed tree expansion is limited to a single-trace setting. In contrast to [FRWJ25], we argue to *not* apply masking to the seed expansion procedure as only freshly sampled data is involved in this step, independent of any (potentially attacker-controlled) input.

4 Hardening the Hypercube ZK Protocol

The zero-knowledge protocol used in Hypercube-SDitH results in all-but-one of the secret party shares from the MPCitH computations being released in the signature. As described in Section 3.2, this means an attacker can reveal the (unshared) long-term secret through a single ‘probe’ on the final (hidden) share. We now present a hardened version of the hypercube zero-knowledge protocol that introduces a *slack in revealed shares*, increasing the number of hidden shares from one to $k + 1$ to resist k -order DPAs. The resulting hardened ZK protocol can be used to build more efficient to protect MPCitH signature schemes utilizing the inherent side-channel protection provided by the secret sharing.

Slack in Revealed Parties The introduction of a slack in revealed parties has been proposed and discussed in the literature for several other variants of MPC-in-the-Head signature schemes [SBWE20, FRWJ25]. Instead of revealing all-but-one $(n - 1)$ party shares, an additional number of k shares stay hidden leading to a total of $k + 1$ hidden shares. As a result, each additional hidden share would have to be probed by an attacker in order to model the power consumption in a DPA attack. More generally, $k + 1$ hidden shares will provide k -order DPA protection for the share expansion and commitment as well as the party computations.

4.1 Our Tweaked $(k+1)$ -Hypercube ZK Protocol

Our tweaked $(k + 1)$ -Hypercube ZK Protocol described in Algorithm 1 largely conforms to the original Hypercube ZK Protocol. The N^D leaf shares are generated using a GGM tree and accumulated to form D sets of N main shares using which $(N - 1) \cdot D$ party computations are performed. However, instead of revealing $N^D - 1$ shares to the verifier by selecting a single hidden leaf share, a number of $k + 1$ hidden leaf shares are selected by the verifier and the remaining $N^D - (k + 1)$ leaf shares are revealed by the prover.

The side N of the hypercube determines the number of shares per dimension and thus the maximum number of hidden main shares per dimension μ denoted as $k'_\mu \leq N$. In this setting, the dimension D determines the soundness error of choosing $n - (k + 1)$ shares to reveal in a single repetition of the MPC protocol, thus the soundness error, $\varepsilon = \frac{k+1}{N^D}$. Interestingly, for a fixed N , the choice of D (and $\#$ parallel MPC repetitions τ) allows for a trade-off between computation costs and communication costs (i.e., signature size for the use in a signature scheme) [AMGH⁺23, Section 5.2].

Algorithm 1: $(k+1)$ -Hypercube ZK Protocol (SD-based)

Input: The prover knows the witness x and derives polynomials Q, P, S . Both parties have the public key (H, y) . There exists a proof Π that outputs ACCEPT if the witness is valid.
Round 1: Prover generates shares and performs commitment.
 Sample the leaf shares $\llbracket s \rrbracket_{i \in [1, n]}$, where $s = (P, Q, S, a, b, c)$.
 Set the auxiliary $\text{aux} = s - \sum_{i=1}^{n-1} \llbracket s \rrbracket_i$.
 Commit to all leaf shares $\llbracket s \rrbracket_i$.
Round 2: Verifier generates randomness $\text{CH}_1 = (r, \varepsilon)$ for the MPC protocol.
Round 3: Prover emulates the MPC protocol.
 Construct the main party shares $\llbracket s \rrbracket_{\mu, \nu}$ from the set of leaf shares with index in $\mathcal{I}_{\mu, \nu} = \{i = (i_1, \dots, i_D) \in [1, N]^D \mid (i_\mu = \nu)\}$ i.e. the leaf share's μ -th hypercube coordinate being ν as $\llbracket s \rrbracket_{\mu, \nu} = \sum_{i \in \mathcal{I}_{\mu, \nu}} \llbracket s \rrbracket_i$.
 Perform the plain computation $(v, \alpha, \beta) = \Pi_{\text{plain}}(s)$ using the randomness (r, ε) .
 Perform party computation $(\llbracket v \rrbracket, \llbracket \alpha \rrbracket, \llbracket \beta \rrbracket)_{\mu, \nu} = \Pi(\llbracket s \rrbracket_{\mu, \nu})$ on each main share $\llbracket s \rrbracket_{\mu, \nu}$ using the randomness (r, ε) .
 Commit to $v, \alpha, \beta, (\llbracket v \rrbracket, \llbracket \alpha \rrbracket, \llbracket \beta \rrbracket)_{\mu, \nu}$ and used randomness (r, ε) .
Round 4: Verifier generates random view challenge $\text{CH}_2 = I^*$.
 Pick $(k + 1) = \prod_{\mu \in [1, D]} k'_\mu$ leaf party shares indices $I^* = \{i_1^*, \dots, i_{k+1}^*\}$ to remain hidden.
Round 5: Prover opens view of selected leaf parties.
 Reveal $\llbracket s \rrbracket_i$ for $n - (k + 1)$ non-hidden leaf parties where $i \notin I^*$.
 Reveal α, β, v and $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu}$ where $1 < k'_\mu < N$.
Verification: Verifier reconstructs fully revealed main shares and recomputes $(\llbracket v' \rrbracket, \llbracket \alpha' \rrbracket, \llbracket \beta' \rrbracket)_{\mu, \nu} = \Pi(\llbracket s \rrbracket_{\mu, \nu})$.
 Accepts if:
 Revealed $n - (k + 1)$ leaf party shares $\llbracket s \rrbracket_i$ are consistent with the committed shares.
 Recomputed $(\llbracket v' \rrbracket, \llbracket \alpha' \rrbracket, \llbracket \beta' \rrbracket)_{\mu, \nu}$ are consistent with committed $(\llbracket v \rrbracket, \llbracket \alpha \rrbracket, \llbracket \beta \rrbracket)_{\mu, \nu}$.
 Revealed non-recomputable $(\llbracket v \rrbracket, \llbracket \alpha \rrbracket, \llbracket \beta \rrbracket)_{\mu, \nu}$ are consistent with committed $(\llbracket v \rrbracket, \llbracket \alpha \rrbracket, \llbracket \beta \rrbracket)_{\mu, \nu}$.
 Proof Π performed by the prover outputs ACCEPT.

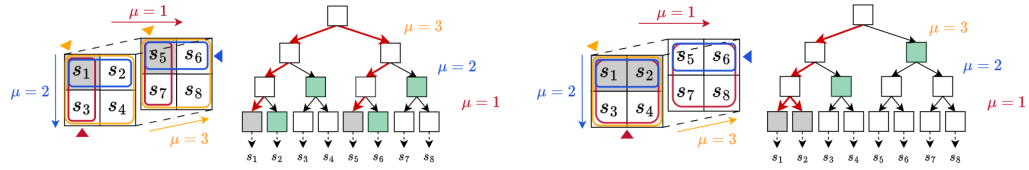
4.2 Optimal Selection of $k + 1$ Hidden Shares in Hypercube MPCitH

We recall from Section 2.4 that the hypercube optimization enables the geometric re-arranging of the $n = N^D$ leaf shares to D sharings between N main parties, reducing the amount of party computations. As described in Section 3.1, leaf shares are expanded from a root seed using a GGM tree which allows for optimized revealing of $k + 1$ share. When using a GGM tree to reveal all-but- $(k + 1)$ shares, instead of directly revealing $N^D - (k + 1)$ leaf seeds, all sibling seeds along the $k + 1$ hidden paths (i.e. the paths between each leaf seed and the root seed) are revealed when they are not part of any hidden path. Thus, the number of sibling seeds to reveal grows with both the number of hidden seeds and the level at which the hidden paths merge (in the worst case, at the root seed).

General Case In our tweaked $(k + 1)$ -Hypercube ZK Protocol, not one but k'_μ main shares are randomly selected and kept hidden for each dimension μ of the hypercube. This results in a total number of $(k + 1) = \prod_{\mu=1}^D k'_\mu$ hidden leaf shares and yields a k -order probing secure ZK protocol. In this setting, the number of hidden leaf shares is restricted to any choice of $k + 1$ that can be factored into D factors each smaller than or equal to N . For a hypercube of $N = 2$ and $D = 3$, this method can be used to select $(k + 1) = \{1, 2, 4\}$ hidden shares. Figure 3a depicts this selection strategy for $(k + 1) = 2$ with $k'_1 = 2$ (red), $k'_2 = 1$ (blue) and $k'_3 = 1$ (orange). As can be seen from the corresponding tree, the naïve approach for selecting shares can result in shares belonging to different subtrees (in

gray), resulting in a large number of sibling nodes (green) that must be made public in the signature to be able to fully reconstruct the tree.

Further overhead occurs for use in a signature scheme, where the plain broadcast values (α, β, v) and the broadcast values of the MPC $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu}$ are not by default included in the signature and instead of verifying each recomputed broadcast value, only the hash value of all commitments in Round 3 (Algorithm 1) is verified. For dimensions μ with $k'_\mu = 1$, the verifier can compute the single partially-revealed (i.e. not-reconstructable) main parties' broadcast value as $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu'} = (\alpha, \beta, v) - \sum_{\nu \neq \nu'} (\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu}$. Dimensions with $k'_\mu = N$ can be entirely skipped during verification, as not a single main share is fully revealed. For dimensions with $1 < k'_\mu < N$ however, the prover must provide $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu}$ for $k'_\mu - 1$ partially-revealed main share per dimension μ in the signature.



(a) General case for selection of $k+1=2$ shares with $k'=(1,1,2)$. Hiding 2 shares (gray) requires revealing 4 seeds (green). (b) Optimized Selection of $k+1=2^l=2$ shares from subtree of height $l=1$. Hiding 2 shares (gray) requires revealing only 2 seeds (green).

Figure 3: Selection of $k+1$ hidden shares on hypercube with $N=3$ and $D=2$

(Optimized) Selection of $k+1$ Leaf Shares from Subtree We observe that this communication cost can be reduced by using the GGM tree structure, and hiding a l -level binary subtree such that the $k+1$ hidden paths merge at the lowest possible level. As a result, the number of hidden shares is restricted to powers of 2. Hiding the subtree makes the number of revealed seeds per round again constant and allows for a reduction compared to both the general case (for both average and worst cases) and the original hypercube protocol. In a hypercube of side $N=2$ and dimension $D=3$, using a 3-level binary seed tree, this method can be used to select $(k+1) = \{2, 4\}$ hidden leaf shares. In Figure 3b, this selection strategy is shown for a subtree of height $l=1$ as a result of hiding $k'_3=1$ (orange), $k'_2=1$ (blue) and all $k'_1=3$ (red). As can be seen in the tree, higher-level sibling nodes (green) suffice to be able to reconstruct the tree, as all hidden leaf shares (gray) are grouped.

For use in the signature scheme, the overhead is further reduced, as by design for all dimensions μ either $k'_\mu = 1$ or $k'_\mu = N$, which does not require the prover to provide any broadcast values $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu}$ in the signature. Moreover, it also allows to reduce the amount of necessary MPC computations, as dimensions with $k'_\mu = N$ and thus no fully-revealed main share do not contribute to the soundness and can therefore be excluded from both the proof construction and verification.

4.3 Security Proof

We now provide a formal security proof for completeness, soundness and the zero-knowledge property of the hardened $(k+1)$ -Hypercube ZK Protocol. Our proofs are structured similarly to the initial SDitH proposal [FJR22] and its hypercube variant [AMGH⁺23]. We first show the (perfect) completeness of Algorithm 1, i.e. any honest prover performing the ZKP will be accepted by an honest verifier with certainty. Contrary to that, we show by proving the soundness that any prover without knowledge of the secret key will only

be accepted with a probability lower than $\varepsilon \approx \frac{k+1}{N^D}$. Finally, we prove that the ZKP is zero-knowledge against an honest verifier that follows the protocol.

Definition 1 (Consistency of views, [IKOS09]). Two views view_i and view_j constructed during execution of the MPC protocol Π are consistent, if the outgoing messages implicit in view view_i are identical to the incoming messages in view_j and vice versa.

Consistency with respect to the MPC protocol Π means that the view is constructed following protocol Π , which can be checked by the verifier by reconstructing an execution of Π for the opened views. Consistency in respect to the public input x means that the opened shares indeed come from a secret sharing of x , which can be checked by checking consistency with the commitments.

Lemma 1 (Lemma 2.3, [IKOS09]). *Let Π be an n -party protocol between the parties $P_1 \dots P_n$ with the public input x and $\{\text{view}_1, \dots, \text{view}_n\}$ be a set of (possibly incorrect) views. Then all views view_i and view_j are pairwise consistent with each other in respect to Π and x if and only if there exists an honest execution of Π in which view_i is the view of P_i .*

Theorem 1 (Completeness). *An honest prover $\mathcal{P}$ with the knowledge of the witness w , who follows the protocol defined in Algorithm 1 will be accepted by an honest verifier $\mathcal{V}$ with a probability of 1.*

Proof. An honest verifier $\mathcal{V}$ will accept a transcript T of an execution of the protocol defined in Algorithm 1 if the revealed views are consistent as defined in Lemma 1. In an honest execution a valid secret sharing of a witness w such that $\sum_{i=1}^n w_i = w$ is used and the MPC protocol Π is followed by all parties. Therefore it will always produce a transcript with consistent views which will be accepted by $\mathcal{V}$ with a probability of 1. $\square$

Lemma 2. *A prover $\tilde{\mathcal{P}}$ that commits to a bad witness $S \cdot Q \neq P \cdot F$ and cannot find a hash collision has a probability of*

$$p + (1 - p) \cdot \varepsilon_{\text{cheat}} = \begin{cases} p + (1 - p) \cdot \frac{k+1}{N^D} & \text{General case} \\ p + (1 - p) \cdot \frac{k+1}{2^D} & \text{Hidden subtree} \end{cases}$$

of being accepted by the verifier $\mathcal{V}$.

Proof. A malicious prover $\tilde{\mathcal{P}}$ without knowledge of a valid set of polynomials such that $S \cdot Q = P \cdot F$ can convince an honest verifier $\mathcal{V}$ to accept if either: (i) with a false positive probability of p , for all challenge points the sharing $\llbracket v \rrbracket$ encodes the zero vector; or (ii) $\tilde{\mathcal{P}}$ cheats on the communications, so it appears that $\llbracket v \rrbracket$ encodes the zero vector.

During the verification, $\mathcal{V}$ checks the consistency of the views of the main parties. To detect cheating by $\tilde{\mathcal{P}}$ at any point in the protocol, it is sufficient for $\mathcal{V}$ to detect cheating in one of the dimensions. As $\mathcal{V}$ chooses a number of k' main shares to hide, such that it receives the views of the remaining main shares in the protocol, independently for each dimension consider the case where $D = 1$. With a probability of $(1 - p)$, $\tilde{\mathcal{P}}$ has to cheat on the communications $\llbracket \alpha \rrbracket$, $\llbracket \beta \rrbracket$ or $\llbracket v \rrbracket$ of at least one of the main parties to make them encode the zero vector ($v = 0$). Note that the underlying shares for each of the MPCitH protocols are identical between the dimensions, therefore the main shares of each dimension encode the same plain value. Hence for $\tilde{\mathcal{P}}$ to cheat it is sufficient to consider that they cheat on at least one of the leaf shares, s_i .

Consider the case where $\tilde{\mathcal{P}}$ cheats on δ leaf shares in total. Since $\mathcal{V}$ chooses k' main shares to hide for each dimension, the probability that $\mathcal{V}$ finds at least one of the hidden shares among the $N - k'$ views of main shares, therefore detecting cheating, is higher if the δ corrupted shares are distributed among more of the N main shares. Therefore, $\tilde{\mathcal{P}}$ will

always attempt to consolidate the corrupted shares in as few main shares as possible. Hence, for the worst case analysis, we may assume that all δ shares lie in 1 main share.

Moreover, $\tilde{\mathcal{P}}$ is not bound to the leaf shares it has corrupted for each execution of the MPCitH protocol, i.e. each dimension, hence for the optimal chance of convincing the verifier, $\tilde{\mathcal{P}}$ would corrupt shares in each of the dimension such that the above consolidation is obtained. Therefore, we may assume that all δ lie in 1 main share for all dimensions.

General case. As $\mathcal{V}$ chooses k' shares uniformly randomly, and $\tilde{\mathcal{P}}$ has corrupted 1 main share, the probability that $\tilde{\mathcal{P}}$ is not detected is given by the probability that of all k' out of N shares $\mathcal{V}$ chooses, $\mathcal{V}$ does not choose the one share $\tilde{\mathcal{P}}$ has corrupted. Hence,

$$\varepsilon_{\text{cheat, dim}} = \frac{\binom{N-1}{k'-1}}{\binom{N}{k'}} = \frac{k'}{N}$$

$\mathcal{V}$ chooses k'_μ shares for each dimension $\mu \in [1, D]$ under the condition that the total number of hidden leaf shares is $k+1$, hence for all dimensions, the probability that $\tilde{\mathcal{P}}$ is not detected is

$$\varepsilon_{\text{cheat}} = \frac{\prod_{\mu=1}^D k'_\mu}{N^D} = \frac{k+1}{N^D}$$

(Optimized) Selection of $k+1$ Leaf Shares from Subtree. Consider a binary tree with $2^l \geq |\{s_i\}_i|$ leaves. The height of the hidden subtree is given by $l = \log(k+1)$. Each level of the binary tree is equivalent to a dimension in a hypercube of side 2, i.e. with in each dimension $\mathcal{V}$ chooses half of the shares to see halving the number of hidden shares in each dimension. Hence,

$$\varepsilon_{\text{cheat, dim}} = \frac{1}{2}$$

Since we can do this for at most $D-l$ dimensions the resulting error is given by

$$\varepsilon_{\text{cheat}} = \frac{1}{2^{D-l}} = \frac{2^l}{2^D} = \frac{2^{\log(k+1)}}{2^D} = \frac{k+1}{2^D}$$

This results in an upper bound of

$$\text{error} = p + (1-p) \cdot \varepsilon_{\text{cheat}} = \begin{cases} p + (1-p) \cdot \frac{k+1}{N^D} & \text{General case} \\ p + (1-p) \cdot \frac{k+1}{2^D} & \text{Hidden subtree} \end{cases}$$

□

Lemma 3 (Splitting Lemma [PS00], as in [FJR22, AMGH⁺23]). *Suppose $S \subset \mathcal{U} \times \mathcal{V}$ satisfies $\Pr[(u, v) \in S] \geq \varepsilon$. Define $T := \{(u, v) \in \mathcal{U} \times \mathcal{V} \mid \Pr_{v' \in \mathcal{V}}[(u, v') \in S]\} \geq (1-\alpha) \cdot \varepsilon$; for any $\alpha \in [0, 1]$. Then the following hold: $\Pr[T] \geq \alpha \cdot \varepsilon$ and $\Pr[T|S] \geq \alpha$.*

Theorem 2 (Soundness). *If a prover $\tilde{\mathcal{P}}$ without the knowledge of a witness is capable of convincing a verifier $\mathcal{V}$ with a probability of $\tilde{\varepsilon} > \varepsilon$ there exists an extraction function E , where the average number of calls to $\tilde{\mathcal{P}}$ is upper bounded by an expression in $\text{poly}(\lambda, (\tilde{\varepsilon} - \varepsilon)^{-1})$.*

In the following, we prove that if a prover $\tilde{\mathcal{P}}$ can convince a verifier $\mathcal{V}$ with a probability greater than the soundness error ε , an extractor E can efficiently extract the witness which in turn is equivalent to $\tilde{\mathcal{P}}$ knowing the witness. Cheating with a probability smaller than the soundness error ε is not an issue for the soundness of the ZKP given ε is reasonably small.

Extraction condition. As defined in the original security proof for the hypercube framework [AMGH⁺23], the extraction of the witness is possible given the opened state and communication for all parties under the additional condition that each party has to be verified in

at least one accepting transcript. In our hardened hypercube, this condition is fulfilled whenever all main party shares for one dimension are opened in at least one transcript. Given a first challenge $I^* = (i_1^*, \dots, i_D^*)$ with $i_\mu^* = \{i_{\mu,1}^*, \dots, i_{\mu,k'_\mu}^*\}$ consisting of k'_μ hidden main shares per dimension μ this would require a maximum number of $\min(k'_\mu)$ additional accepting transcripts. As we however want to relate the probability of extraction to the soundness error ε in the course of our proof, we pick a slightly stricter extraction condition which implies the one previously defined. For a first transcript with the challenge I^* and a total of $k + 1$ hidden leaf shares, we require a maximum number of $k + 1$ additional accepting transcripts. For each hidden leaf share in I^* at least one transcript must be found, where this leaf share is opened. This means that successful extraction requires collecting $k + 1$ additional successful transcripts, where each transcript must fulfill the described *partial extraction condition*.

Proof. In the following we assume the commitment scheme to be perfectly binding (instead of computationally binding), meaning the same commitments com can only be produced by the same shares. For accepting transcripts which share their initial commitments to the leaf shares $h = \text{hash}(\text{com}_1, \dots, \text{com}_{ND})$ but differ in the reveal challenge that means either $\llbracket Q \rrbracket$, $\llbracket S \rrbracket$ and $\llbracket P \rrbracket$ are not consistent with the revealed shares and we found a hash collision or the revealed shares are consistent (i.e. the openings are unique) and $\llbracket Q \rrbracket$, $\llbracket S \rrbracket$ and $\llbracket P \rrbracket$ are uniquely defined. In the following we only focus on the extractor finding consistent shares. A witness $(\llbracket Q \rrbracket, \llbracket S \rrbracket, \llbracket P \rrbracket)$ is defined to be a *good witness* if $Q \cdot S = P \cdot F$.

We define an extractor E that runs the prover $\tilde{\mathcal{P}}$ using a randomness R_h for the generation of leaf shares in the initial commitment. r_h is a possible realization of R_h . The extractor E performs the following steps:

1. Run $\tilde{\mathcal{P}}$ with an honest verifier $\mathcal{V}$ and any R_h until a successful transcript T_0 is found using a specific randomness r_h . This transcript has a challenge $I^* = \{i_1^*, \dots, i_{k+1}^*\}$ with a total of $k + 1$ hidden leaf shares.
2. Rewind $\tilde{\mathcal{P}}$ using the same randomness r_h until a successful transcript T_κ with a reveal challenge $J^* = \{j_1^*, \dots, j_{k+1}^*\}$ is found, such that the κ -th hidden leaf share in transcript T_0 is not hidden in T_κ . Repeat this for all $k + 1$ hidden leaf shares in T_0 .
3. Extract the witness, if the witness is *bad* start again with step 1.

We now estimate the average number of calls to $\tilde{\mathcal{P}}$ the extractor E has to perform to extract the witness successfully. In the following, $\text{succ}_{\tilde{\mathcal{P}}}^{T_\kappa}$ denotes the event in which the prover $\tilde{\mathcal{P}}$ successfully convinces $\mathcal{V}$ in transcript T_κ . We also use $\text{succ}_{\tilde{\mathcal{P}}}$ for the general event of the prover succeeding, independent of any specific transcript.

According to our hypothesis, the prover can find a successful transcript with a probability of $\Pr[\text{succ}_{\tilde{\mathcal{P}}}] = \tilde{\varepsilon}$ on average. To apply the Forking Lemma, we first have to show that the set of random variables for which $\tilde{\mathcal{P}}$ succeeds exists with a non-negligible probability. We therefore define r_h is *good* if $\Pr[\text{succ}_{\tilde{\mathcal{P}}} \mid R_h = r_h] \geq (1 - \alpha) \cdot \tilde{\varepsilon}$ where $\alpha \in (0, 1)$. Using the Splitting Lemma (Lemma 3) we can derive $\Pr[r_h \text{ good} \mid \text{succ}_{\tilde{\mathcal{P}}}] \geq \alpha$. That implies good randomness can be found after approximately $\frac{1}{\alpha}$ successful transcripts. Assuming r_h is good, by Lemma 2 we can say $\Pr[\text{succ}_{\tilde{\mathcal{P}}} \mid R_h = r_h] \geq (1 - \alpha) \cdot \tilde{\varepsilon} > \varepsilon$ which implies the witness encoded is a good witness, as for a bad witness $\Pr[\text{succ}_{\tilde{\mathcal{P}}} \mid R_h = r_h] \leq \varepsilon$.

To proceed, we want to bound the number of required calls to $\tilde{\mathcal{P}}$ until a transcript T_κ is found that fulfills the partial extraction condition $\text{PExt}_\kappa := \{i_\kappa^* \notin J^*\}$ i.e. the κ -th hidden leaf share in T_0 is not hidden in T_κ . We assume a good randomness r_h has been found and use the counter probability of not finding a transcript T_κ that fulfills PExt_κ because the specific hidden leaf share in T_0 is again hidden in T_κ . This probability is equivalent to the probability $\varepsilon_{\text{cheat}}$ described in Lemma 2 with which an initially corrupted leaf share is not

detected by $\mathcal{V}$, because this specific leaf share is hidden. As a result, a lower bound for the probability of finding a transcript that fulfills PExt_κ in every step is found in Equation 3.

$$\begin{aligned} \Pr[\text{succ}_{\tilde{\mathcal{P}}}^{T_\kappa} \cap \text{PExt}_\kappa \mid r_h \text{ good}] &= \Pr[\text{succ}_{\tilde{\mathcal{P}}}^{T_\kappa} \mid r_h \text{ good}] - \Pr[\text{succ}_{\tilde{\mathcal{P}}}^{T_\kappa} \cap \neg \text{PExt}_\kappa \mid r_h \text{ good}] \\ &\geq (1 - \alpha) \cdot \tilde{\varepsilon} - \Pr[\neg \text{PExt}_\kappa \mid r_h \text{ good}] = (1 - \alpha) \cdot \tilde{\varepsilon} - \Pr[\neg \text{PExt}_\kappa] \\ &\geq (1 - \alpha) \cdot \tilde{\varepsilon} - \varepsilon_{\text{cheat}} \geq (1 - \alpha) \cdot \tilde{\varepsilon} - \varepsilon \end{aligned} \quad (3)$$

With this probability and the same good randomness r_h as for T_0 , the probability of obtaining a transcript T_κ that fulfills PExt_κ is greater $\frac{1}{2}$ after a number of N_κ calls to $\tilde{\mathcal{P}}$.

$$N_\kappa \approx \frac{\ln(\frac{1}{2})}{\ln(1 - ((1 - \alpha) \cdot \tilde{\varepsilon} - \varepsilon))} = \frac{\ln(2)}{\ln(\frac{1}{1 - ((1 - \alpha) \cdot \tilde{\varepsilon} - \varepsilon)})} \leq \frac{\ln(2)}{(1 - \alpha) \cdot \tilde{\varepsilon} - \varepsilon} \quad (4)$$

Let C denote the number of calls to $\tilde{\mathcal{P}}$ made by the extractor. We denote the expected value of calls as $\mathbb{E}[C]$ in a recursive formula, considering the scenario of failing a step and starting a new iteration of the outer loop with certain probabilities. For one iteration:

- The extractor makes one call to $\tilde{\mathcal{P}}$ to obtain T_0 .
- If T_0 is not successful, which occurs with a probability of $1 - \Pr[\text{succ}_{\tilde{\mathcal{P}}}]$:
 - The extractor continues to the next iteration and makes an average of $\mathbb{E}[C]$ calls to $\tilde{\mathcal{P}}$.
- If T_0 is successful, which occurs with a probability of $\Pr[\text{succ}_{\tilde{\mathcal{P}}}]$:
 - The extractor tries to link $k + 1$ transcripts for $\kappa = [1, k + 1]$:
 - If r_h is good, which occurs with a probability of α , in total the extractor makes at most $(k + 1) \cdot N_\kappa$ calls to $\tilde{\mathcal{P}}$ and returns a transcript T_κ that fulfills PExt_κ with a probability of $\frac{1}{2^{k+1}}$.
 - Otherwise, with a probability of at most $1 - \frac{\alpha}{2^{k+1}}$ the extractor makes at most $(k + 1) \cdot N_\kappa$ calls to $\tilde{\mathcal{P}}$ and continues with a new iteration of the outer loop making an average of $\mathbb{E}[C]$ calls to $\tilde{\mathcal{P}}$.

$$\mathbb{E}[C] \leq 1 + \underbrace{(1 - \Pr[\text{succ}_{\tilde{\mathcal{P}}})) \cdot \mathbb{E}[C]}_{T_0 \text{ not found}} + \underbrace{\Pr[\text{succ}_{\tilde{\mathcal{P}}}] \cdot ((k + 1) \cdot N_\kappa + \overbrace{(1 - \frac{\alpha}{2^{k+1}}) \cdot \mathbb{E}[C]})^{T_1 \dots T_\kappa \text{ not found}}}_{T_0 \text{ found}} \quad (5)$$

$$\begin{aligned} \mathbb{E}[C] &\leq 1 + (1 - \tilde{\varepsilon}) \cdot \mathbb{E}[C] + \tilde{\varepsilon} \cdot ((k + 1) \cdot N_\kappa + (1 - \frac{\alpha}{2^{k+1}}) \cdot \mathbb{E}[C]) \\ &\leq \frac{1 + \tilde{\varepsilon} \cdot (k + 1) \cdot N_\kappa}{\tilde{\varepsilon} - \tilde{\varepsilon} \cdot (1 - \frac{\alpha}{2^{k+1}})} = \frac{2^{k+1}}{\tilde{\varepsilon} \cdot \alpha} \cdot (1 + \tilde{\varepsilon} \cdot (k + 1) \cdot N_\kappa) \\ &\leq \frac{2^{k+1}}{\tilde{\varepsilon} \cdot \alpha} \cdot (1 + \tilde{\varepsilon} \cdot (k + 1) \cdot \frac{\ln(2)}{(1 - \alpha) \cdot \tilde{\varepsilon} - \varepsilon}) \end{aligned}$$

We choose α to be $(1 - \alpha) \cdot \tilde{\varepsilon} = \frac{1}{2} \cdot (\varepsilon + \tilde{\varepsilon})$ and $\alpha = \frac{1}{2} \cdot (1 - \frac{\varepsilon}{\tilde{\varepsilon}})$ yielding the upper bound in Equation 6.

$$\mathbb{E}[C] \leq \frac{2^{k+2}}{\tilde{\varepsilon} - \varepsilon} \cdot (1 + \frac{2 \cdot \tilde{\varepsilon} \cdot (k + 1) \cdot \ln(2)}{\tilde{\varepsilon} - \varepsilon}) \quad (6)$$

□

Theorem 3 (Honest-Verifier Zero Knowledge). *Given a $(t, \varepsilon_{\text{PRG}})$ -secure PRG and a $(t, \varepsilon_{\text{com}})$ -hiding commitment scheme, there exists an efficient simulator $\mathcal{S}$, which gets as input the two challenges (r, ε) and $I^* = \{i_1^*, \dots, i_{k+1}^*\}$ and outputs a transcript such that it is indistinguishable from a real transcript of the protocol defined in Algorithm 1.*

Proof. In the following, we demonstrate that the simulator $\mathcal{S}_{v3}$ produces transcripts with a distribution that is indistinguishable from the distribution of real transcripts. We start with a simulator $\mathcal{S}_{v0}$ that produces real transcripts and adapt the algorithm step-by-step until we arrive at the algorithm $\mathcal{S}_{v3}$. For each adaptation, we argue why the distribution of simulators transcripts is unchanged in the previous version.

Algorithm 2: HVZK Simulator $\mathcal{S}_{v3}$

```

Randomly sample root seed rseed  $\xleftarrow{\$} \{0, 1\}^\lambda$ .
Step 1 (Sample Challenges):
Sample  $\text{CH}_1 = \{r, \varepsilon\} \xleftarrow{\$} \mathbb{F}_{\text{points}}^t \times \mathbb{F}_{\text{points}}^t$  and  $\text{CH}_2 = I^* = (i_1^*, \dots, i_D^*)$  with  $i_\mu^* \xleftarrow{\$} ([N]_{k'})$ .
Step 2 (Generate Leaf Party States):
Expand root seed rseed to get  $N^D$  leaf states  $\text{seed}_i$ .
Step 3 (Generate Leaf Party Commitments and Witness Shares):
for  $i \notin I^*$  do
    if  $i \neq N^D$  then
        Compute  $\text{com}_i = \text{Hash}(\text{seed}_i)$ .
        Expand  $\text{seed}_i$  into witness shares  $\llbracket s \rrbracket_i$ .
    else
        Generate aux for the last party  $i = N^D$ , randomly sample  $(\llbracket P \rrbracket, \llbracket Q \rrbracket, \llbracket S \rrbracket, \llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)_{N^D}$ .
        Compute  $\text{com}_{N^D} = \text{Hash}(\text{seed}_i || \text{aux})$ .
for  $i \in I^*$  do
    Randomly sample  $\text{com}_i$ .
    Randomly sample  $(\llbracket P \rrbracket, \llbracket Q \rrbracket, \llbracket S \rrbracket, \llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)_i$ .
Compute  $\text{COM} = \text{Hash}(\text{com}_1, \dots, \text{com}_{N^D})$ .
Step 4 (Generate Party Communications):
for  $(\mu, \nu) \in [1, D] \times [1, N]$  do
    if  $\nu \in i_\mu^*$  then
        Randomly sample  $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu}$ , except for one  $\llbracket v \rrbracket_{\mu, \nu'}$  per dimension which is set
         $\llbracket v \rrbracket_{\mu, \nu'} = - \sum_{\tilde{\nu} \neq \nu'} \llbracket v \rrbracket_{\mu, \tilde{\nu}}$  where  $\tilde{\nu} \in [1, N]$ .
    else
        Compute  $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu}$  as stated in Algorithm 1.
    Compute  $h_{\mu, \nu} \leftarrow \text{Hash}((\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu})$ .
Step 5 (Output transcript):
 $\text{RSP}_1 = \text{Hash}(h_{1,1}, \dots, h_{D,N})$ 
 $\text{RSP}_2 = (\text{com}_{i \in I^*}, (\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu \in i_\mu^*}, \{\text{state}_i\}_{i \notin I^*})$ 
Output:  $(\text{COM}, \text{CH}_1, \text{RSP}_1, \text{CH}_2, \text{RSP}_2)$ 

```

Simulator $\mathcal{S}_{v0}$ (True transcripts): To simulate true transcripts, $\mathcal{S}_{v0}$ takes as input a witness x_A as well as the honest verifiers challenges $\text{CH}_1 = \{r, \varepsilon\}$ and $\text{CH}_2 = I^*$. It executes Algorithm 1 correctly and produces transcripts with a distribution identical to real transcripts.

Simulator $\mathcal{S}_{v1}$: The seed-derived randomness of leaf shares $(\llbracket P \rrbracket, \llbracket Q \rrbracket, \llbracket S \rrbracket, \llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)_{i \in I^*}$ is replaced by true randomness. If the challenge $I^* = \{i_1^*, \dots, i_{k+1}^*\}$ contains the last leaf share index ($N^D \in I^*$), $(\llbracket P \rrbracket, \llbracket Q \rrbracket, \llbracket S \rrbracket, \llbracket c \rrbracket)_{N^D}$ are still generated in the usual way, so the witness main party shares still sum up to the input witness and only shares $\llbracket a \rrbracket_{N^D}$ and $\llbracket b \rrbracket_{N^D}$ are random. In any case, for a challenge $I^* = (i_1^*, \dots, i_D^*)$ the values $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu \in i_\mu^*}$ computed using a true random leaf share are random. Distinguishing the distribution of transcripts from simulator $\mathcal{S}_{v1}$ from $\mathcal{S}_{v0}$ in running time t is bounded by ε_{PRG} .

Simulator $\mathcal{S}_{v2}$: In the next step $(\llbracket S \rrbracket, \llbracket Q \rrbracket, \llbracket P \rrbracket, \llbracket c \rrbracket)_{N^D}$ are also replaced by true randomness. $\mathcal{S}_{v2}$ is now independent of the witness and only takes the challenges CH_1 and CH_2 as input. For $N^D \in I^*$ only $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu \in i_\mu^*}$ are affected in the transcript (because aux is not send). In this case, the output distribution of $\mathcal{S}_{v2}$ is identical to $\mathcal{S}_{v1}$, because those shares are already randomly distributed and independent of other parties in $\mathcal{S}_{v1}$. For $N^D \notin I^*$ also aux is affected in the transcript. In $\mathcal{S}_{v1}$, aux is computed by adding all true random leaf shares $\llbracket s \rrbracket_{i \in I^*}$ and all pseudo-random leaf party shares $\llbracket s \rrbracket_{i \notin I^*}$, therefore generating aux via true randomness does not alter the distribution between $\mathcal{S}_{v1}$ and $\mathcal{S}_{v2}$.

Simulator $\mathcal{S}_{v3}$: In the final step, all $(\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket, \llbracket v \rrbracket)_{\mu, \nu \in i_\mu^*}$ except for one $\llbracket v \rrbracket_{\mu, \nu' \in i_\mu^*}$ per dimension μ are randomly sampled. One $\llbracket v \rrbracket_{\mu, \nu' \in i_\mu^*}$ is set to $\llbracket v \rrbracket_{\mu, \nu'} = -\sum_{(\mu, \nu) \neq (\mu, \nu')} \llbracket v \rrbracket_{\mu, \nu}$. As these already are randomly distributed in $\mathcal{S}_{v2}$, the output distributions of $\mathcal{S}_{v3}$ does not change between $\mathcal{S}_{v2}$ and $\mathcal{S}_{v3}$. The outputs RSP_1 and RSP_2 of $\mathcal{S}_{v3}$ are thus indistinguishable from those of an honest execution of Algorithm 1. When applying the hiding property of the commitment scheme on $\text{com}_{(i_1, \dots, i_D) \in I^*}$, we can conclude that the global HVZK simulator indeed outputs a transcript that is $(t, \varepsilon_{PRG} + \varepsilon_{com})$ -indistinguishable from a real transcript of Algorithm 1. $\square$

5 A Hardened SDitH Digital Signature Scheme

In the following, we present a hardened variant of the SDitH signature scheme, that builds on the $(k+1)$ -Hypercube ZK Protocol proposed in Section 4 and uses binary GGM trees to efficiently reveal all-but- $(k+1)$ shares from a subtree. The side-channel protection provided by our tweaked Hypercube-SDitH signature scheme applies to all parts of the scheme that operate on shares of the secret, i.e. the share expansion from the leaf seeds, the commitment of the leaf shares, the construction of the main shares and the MPC computations. Those parts of the signature scheme benefit from inherent k -th order probing security. Our tweak does not yield additional side-channel protection for the seed expansion via the GGM tree and the plain broadcast computation. The GGM seed expansion however, is inherently resistant to DPA, since no long-term secrets are present in this procedure. The plain broadcast computation operates on the plain secret and is not protected against DPA. This makes the application of masking to this operation necessary to achieve DPA protection for the full scheme.

5.1 Parameter Selection: Speed vs. Bandwidth Trade-Off

Our hardened Hypercube-SDitH uses the same SD parameters as proposed in v1.1 of the NIST candidate SDitH signature scheme [MBB⁺25]. For the MPC parameters, we choose a fixed side of $N = 2$, while using 3 different choices of dimension D (with $D = 5$, $D = 8$ and $D = 12$). This is to obtain a tradeoff between fast and small signatures, while keeping the number of leaf shares $n = N^D$ reasonably small for use on resource constrained devices. As will be demonstrated in our performance analysis, a larger amount of leaf shares ($D = 12$) corresponds to a larger GGM tree (and more state generations and commits) but at the same time higher single-round soundness, which means the number of MPC repetitions τ can be reduced and hence also the signature size decreases. On the contrary, a small hypercube dimension D results in more MPC repetitions, with fewer share generations and commits. As a result the computational cost is reduced, at the cost of larger signatures.

Table 1: Choice of τ for $\lambda = 128$, for proposed MPC parameters (N, D) and protection orders k .

n	N	D	k=0	k=1	k=3	k=7
			τ	τ	τ	τ
32	2	5	27	33	44	65
256	2	8	17	20	23	27
4 096	2	12	12	13	14	16

The additional side-channel protection provided by increasing the number of hidden shares comes at the cost of soundness. This is, because hidden shares are not verified and can be corrupted to cheat in the ZK protocol and increasing their number thus increases the chances of a cheating prover being accepted. The number of parallel repetitions τ is chosen

to achieve a total soundness of the ZKP according to the required security level. Further, due to the use of Fiat-Shamir transforms to create a non-interactive signature scheme, the choice of parameters must also provide sufficient security against the forgery attack described by Kales et al. [KZ20]. That requires a choice of τ such that the minimum cost of this attack as described in Equation 7 is greater than 2^λ , where p is the false positive probability of the used proof and ε is the soundness error of a single repetition of the MPC protocol (Section 5.1).

$$\text{cost}_{\text{forge}} := \min_{0 \leq \tau' \leq \tau} \left\{ \frac{1}{\sum_{i=\tau'}^{\tau} \binom{\tau}{i} p^i (1-p)^{\tau-i}} + \left(\frac{1}{\varepsilon}\right)^{\tau-\tau'} \right\} \quad (7)$$

In Table 1, we show the adaptation of τ to reach a security of $\lambda = 128$ (matching the original SDitH scheme), for each of the proposed MPC parameters.

5.2 Performance Estimation and Analysis

In the following, we analyse and compare the performance (speed & signature size) of our novel Hypercube-SDitH signature scheme, which provides k -order hardening by design, against its unprotected and masked Hypercube-SDitH implementation.

5.2.1 Computational Overhead

We first estimate the computational cost of the Hypercube-MPCitH signature scheme, which is mainly attributed to the following operations: the GGM tree expansion, the share generation and the commitments, all three due to their large use of symmetric primitives, and the MPC computations.

Unprotected SDitH The number of required MPC computations for the unprotected original Hypercube-SDitH signature scheme is equal to $((N-1) \cdot D + 1) \cdot \tau$, and the number of share generations and commitments are both equal to $N^D \cdot \tau$.

Masked Hypercube-SDitH In SDitH, the MPC computations only involve linear operations, as can be seen in Eq. 1 - 2. For k -th order masking, that means each of the $(k+1)$ shares can be processed independently requiring approximately $(k+1)$ -times the number of MPC computations during signing. The total estimated cost of the masked MPC computations during signature generation is:

$$\# \text{ MPC computations}_{\text{sign, fully masked}} = ((N-1) \cdot D + 1) \cdot (k+1) \cdot \tau \quad (8)$$

Further, the share generation and commitment also require masking. To avoid the use of a masked hash function, which is potentially very expensive due to the expected quadratic complexity, the authors in [AMGH⁺23] propose the use of $(k+1)$ parallel unmasked seed trees, randomness expansions and $(k+1)$ parallel commitments of shares which comes with a linear overhead of $(k+1)$. In total, the cost of share generations and commitments is $N^D \cdot (k+1) \cdot \tau$.

Our New Hardened Hypercube-SDitH The proposed tweak, which introduces a slack in the revealed parties, only provides inherent side-channel protection for parts of the signature generation (share expansion, commitment and party computations). In order to perform a fair comparison with the fully masked implementation, we also estimate the overhead of masking the remaining operations during signing: the plain broadcast computation. Further, due to the use of height- l fully hidden subtrees, $l = \log_N(k+1)$

fully hidden dimensions can be excluded from the proof construction. In total, this results in an estimated overhead in the number of MPC computations of:

$$\# \text{ MPC computations}_{\text{sign, PBC masked}} = ((N - 1) \cdot (D - \log_N(k + 1)) \cdot \tau + (k + 1) \cdot \tau \quad (9)$$

The cost of share generation and commitments is calculated according to the unprotected case. Generally, for $k = 0$ the computational cost is identical to the unprotected scheme. For higher order protection ($k > 1$), although the cost per round is identical or even slightly reduced, the overall cost is increased due to the higher number of parallel repetitions τ in order to account for the loss in soundness.

5.2.2 Bandwidth Cost

Another important aspect of the performance of signature schemes is the signature size, as it directly impacts communication cost and required bandwidth.

Unprotected SDitH Each signature contains a salt, hash h_2 the auxiliary share in **view**, the broadcasted plain values (**broad_plain**), the released **path** in **view** and commitments. All contributions result in:

$$\text{size}_{\text{byte, sign}} = \frac{\lambda}{2} + \tau \cdot [k_{\text{SD}} + 2w + (2 \cdot d + 1) \cdot 4t + \frac{\lambda}{8} \cdot \log_2(N) + \frac{\lambda}{4}] \quad (10)$$

Masked Hypercube-SDitH The application of classical masking does not impact the signature size, as masked variables are unmasked before being made public in the signature. However, as proposed in [AMGH⁺23], we assume the use of parallel seed trees and commitments to decrease computational overhead. This comes at the cost of a larger signature size as $(k + 1)$ seeds and commitments of the hidden share need to be included in the signature. In total, this results in a signature size of:

$$\text{size}_{\text{byte, masked}} = \frac{\lambda}{2} + \tau \cdot [k_{\text{SD}} + 2w + (2 \cdot d + 1) \cdot 4t + D \cdot \frac{\lambda}{8} \cdot (k + 1) + \frac{\lambda}{4} \cdot (k + 1)] \quad (11)$$

Our New Hardened Hypercube-SDitH The signature size for our hardened Hypercube-SDitH signature scheme can be calculated as shown in Equation 12. As can be seen, $l = \log_2(k + 1)$ fewer seeds are required in the signature if hiding an l -level subtree.

$$\text{size}_{\text{byte}} = \frac{\lambda}{2} + \tau \cdot [k_{\text{SD}} + 2w + (2 \cdot d + 1) \cdot 4t + (D - \log_2(k + 1)) \cdot \frac{\lambda}{8} + \frac{\lambda}{4} \cdot (k + 1)] \quad (12)$$

5.2.3 Comparison

Table 2 shows the comparison of computational cost and bandwidth between the unprotected, fully masked and inherently-hardened Hypercube-SDitH signature scheme, for different orders of protection k and proposed MPC parameters (N, D) . The overhead of the inherently hardened scheme is mainly caused by the increase in parallel repetitions τ to account for the loss in soundness due to the k additional hidden shares. Hence the tweak yields the best results for higher D and thus higher soundness of a single round (at the cost of a higher baseline computational cost). Generally, with growing D the relative overhead of the hardened scheme for a fixed order of protection k decreases, while the estimated overhead of masking is always linear.

Already for $D = 8$, our hardened scheme substantially reduces the overhead of higher-order DPA protection compared to masking, and even further improves with increased hypercube dimension. For $D = 12$, the hardened scheme comes at only +16.7% offline and +25.6% online overhead ($k = 3$), compared to an estimated +300% with masking. Similarly, the signature size increases by approximately +30% (to 8.8 kB) compared to +119% for the masked estimation.

Table 2: Expected signing computational cost & signature size for different orders of protection k and hypercube parameters. The overhead (+%) is compared to the unprotected Hypercube-SDitH scheme ($k = 0$).

Scheme	Param.		k	Offline Cost			Online Cost		Signature Size (byte)	+%
	N	D		# Share Gen.	# Com.	+	# MPC	+		
Hypercube-SDitH [MFG+23]	2	5	0	864	864	-	162	-	12 160	-
	2	8	0	4 352	4 352	-	153	-	8 496	-
	2	12	0	49 152	49 152	-	156	-	6 784	-
Masked Hypercube-SDitH	2	5	1	1 728	1 728	100	324	100	15 184	24.9
			3	3 456	3 456	300	648	300	21 232	74.6
			7	6 912	6 912	700	1 296	700	33 328	174.1
	2	8	1	8 704	8 704	100	306	100	11 216	32.0
			3	17 408	17 408	300	612	300	16 656	96.0
			7	34 816	34 816	700	1 224	700	27 536	224.1
	2	12	1	98 304	98 304	100	312	100	9 472	39.6
			3	196 608	196 608	300	624	300	14 848	118.9
			7	393 216	393 216	700	1 248	700	25 600	277.4
Our Hardened Hypercube-SDitH	2	5	1	1 056	1 056	22.2	198	22.2	15 376	26.5
			3	1 408	1 408	63.0	308	90.1	22 592	85.8
			7	2 080	2 080	140.8	650	301.2	40 624	234.1
	2	8	1	5 120	5 120	17.7	180	17.7	10 304	21.3
			3	5 888	5 888	35.3	230	50.3	12 944	52.4
			7	6 912	6 912	58.8	351	129.4	18 208	114.3
	2	12	1	53 248	53 248	8.3	169	8.3	7 552	11.3
			3	57 344	57 344	16.7	196	25.6	8 800	29.7
			7	65 536	65 536	33.3	272	74.36	11 840	74.5

5.3 Implementation

We also implement the proposed $(k + 1)$ -Hypercube ZK Protocol and hidden-subtree optimization (Section 4) in C and integrate it in the SDitH reference implementation to obtain a hardened Hypercube-SDitH signature scheme. Table 3 compares its performance to the unprotected SDitH reference implementation, measured on an Intel Core i7-8550U (1.80 GHz). Our implementation performs all hash function calls without hardware acceleration or architecture-specific instruction support. As a result, the majority of the computation time is spent in the offline phase.

Overall, the obtained results match our estimations listed above. Increasing the protection order k results in increased sign and verification times, due to the additional repetitions required to compensate for the loss of soundness introduced by the slack countermeasure, and thus also increases the number of rounds that must be verified. As expected, we observe that especially for smaller dimensions D , the signature size increases, while the signing time is relatively low. For higher dimensions, the general signature size is smaller, at the cost of increasing signing and verification times.

5.4 Comparison to Hardened TCitH Signatures

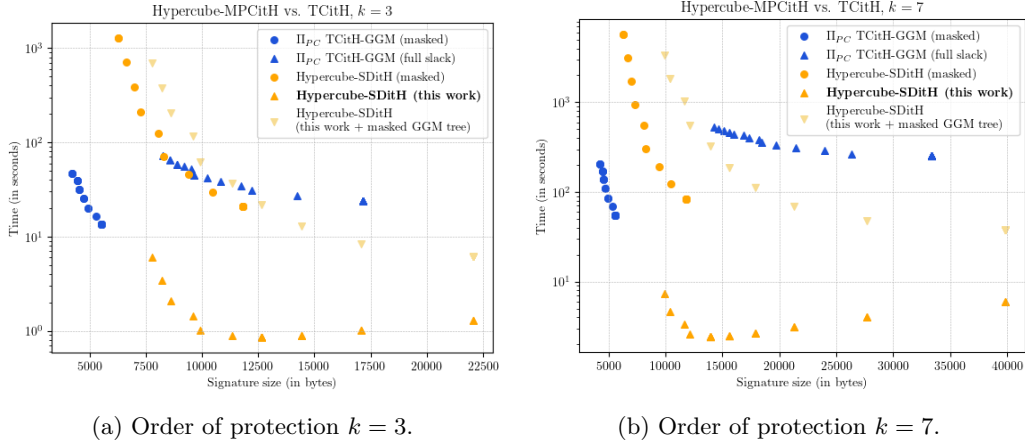
We integrate the proposed Hypercube-SDitH scheme in the framework from [FRWJ25], to estimate its performance on a 32-bit RISC-V platform with hardware acceleration for symmetric primitives. Figure 4 shows estimated signature sizes and signing times for our hardened SDitH signature scheme (yellow) compared to the Π_{PC} TCitH scheme [FR25] (in blue), at protection orders $k = 3$ and $k = 7$ and for different MPC parameters (N, D) .

In comparison to the tweaked (full slack) TCitH-GGM scheme, the hardened Hypercube-SDitH offers significantly faster signing times at comparable signature sizes and more opportunity to explore performance (size/speed) trade-offs. This is partly due to the fact that Feneuil et al. [FRWJ25] recommend to apply masking to the seed expansion, despite it not being vulnerable to DPA (Section 3). We also estimate the cost of a tweaked Hypercube-

Table 3: Benchmarks (time and signature size) for the unprotected and hardened Hypercube-SDitH scheme on Intel Core i7-8550U at 1.80 GHz.

Scheme	Param. N D		k	Sign time (in ms)	Verify time (in ms)	Signature Size (in bytes)
Hypercube-SDitH [MFG+23]	2	5	0	72.22	61.45	12 160
	2	8	0	81.02	73.99	8 496
	2	12	0	263.62	256.71	6 784
This Work	2	5	0	72.75	61.85	12 160
			1	77.31	63.80	15 376
			3	81.87	64.32	22 592
			7	92.40	67.26	40 624
	2	8	0	81.23	74.05	8 496
			1	86.66	78.48	10 304
			3	90.32	80.97	12 944
			7	95.09	83.94	18 208
	2	12	0	266.15	258.45	6 784
			1	280.60	272.64	7 552
			3	293.94	285.34	8 800
			7	330.80	319.97	11 840

MPCitH scheme with masking in the GGM tree seed expansions, especially for large D (i.e. large GGM trees), our proposed Hypercube scheme achieves substantially faster signing time. In conclusion, using our proposed optimizations (unmasked GGM tree expansion and hiding subtrees), Hypercube-MPCitH archives significantly better performance compared to a fully masked implementation, whereas tweaking the TCitH-GGM scheme to achieve inherent DPA protection decreases performance.

Figure 4: Comparison between tweaked TCitH [FRWJ25] ($N = 32$ to $N = 256$) and our hardened Hypercube-SDitH ($n = 2^5$ to $n = 2^{15}$) using the MPCitH estimator.

6 Conclusion

In this work, we first performed an analysis of the side-channel security of an SD-based Hypercube-MPCitH signature scheme, and identified DPA- and SPA targets. We showed that none of the known attacks on MPCitH signature schemes can be applied to mount a DPA on the fully ephemeral parts of the signature scheme, e.g. the GGM tree. Operations on the shares of the secret however, were identified to be vulnerable to DPA attacks. To address this, we introduced a hardened Hypercube-MPCitH zero-knowledge protocol that provides k -th-order DPA resistance without the linear overhead inherent to masking. We

further proposed a structured selection of $k + 1$ hidden shares for reduced signature size. Applied to SDitH, for $n = 2^8$, the hardened signature scheme can achieve higher-order DPA-protection with an overhead of around 18% in computational cost and 21.3% in signature size for 1st-order and 59 – 130% and 114%, respectively, for 7th-order protection.

Acknowledgments

This work has been partially supported by the Horizon 2020 ERC Advanced Grant (101020005 Belfort), the Cybersecurity Research Program Flanders (CRPF) with reference number VOEWICS02, the Belgian-QCI (3E230370) (see beqci.eu), Intel Corporation, the Research-I foundation from Infosys, the Initiation grant from IIT Kanpur, Prime Minister Early Career Research Grant from Anusandhan National Research Foundation, the Google India research fellowship, the Defense Advanced Research Projects Agency (DARPA) under Contract No. HR001120C0085 and by the FWO under an Odysseus project GOH9718N. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of DARPA, the US Government, Cyber Security Research Flanders, or the FWO. The U.S. Government is authorized to reproduce and distribute reprints for governmental purposes, notwithstanding any copyright annotation therein.

References

- [AAC⁺22] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, and Daniel Smith-Tone. Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. Online. Accessed 27th November, 2025, 2022.
- [ABE⁺21] Diego F Aranha, Sebastian Berndt, Thomas Eisenbarth, Okan Seker, Akira Takahashi, Luca Wilke, and Greg Zaverucha. Side-channel protections for picnic signatures. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 239–282, 2021.
- [AEVR23] Guilhem Assael, Philippe Elbaz-Vincent, and Guillaume Reymond. improving single-trace attacks on the number-theoretic transform for cortex-m4. In *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 111–121. IEEE, 2023.
- [AMGH⁺23] Carlos Aguilar-Melchor, Nicolas Gama, James Howe, Andreas Hülsing, David Joseph, and Dongze Yue. The return of the sdith. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 564–596. Springer, 2023.
- [ARS⁺15] Martin R Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for mpc and fhe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 430–454. Springer, 2015.
- [BBD⁺24] Slim Bettaieb, Loïc Bidoux, Victor Dyseryn, Andre Esser, Philippe Gaborit, Mukul Kulkarni, and Marco Palumbi. Perk: compact signature scheme based on a new variant of the permuted kernel problem. *Designs, Codes and Cryptography*, 92(8):2131–2157, 2024.

- [BBdSG⁺23] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from vole-in-the-head. In *Annual international cryptology conference*, pages 581–615. Springer, 2023.
- [BCDF⁺25] Loïc Bidoux, Jesús-Javier Chi-Domínguez, Thibault Feneuil, Philippe Gaborit, Antoine Joux, Matthieu Rivain, and Adrien Vinçotte. Ryde: a digital signature scheme based on rank syndrome decoding problem with mpc-in-the-head paradigm. *Designs, Codes and Cryptography*, pages 1–36, 2025.
- [BDT⁺23] Yaacov Belenky, Ira Dushar, Valery Teper, Vadim Bugaenko, Oleg Karavaev, Leonid Azriel, and Yury Kreimer. Carry-based differential power analysis (cdpa) and its application to attacking hmac-sha-2. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):1–29, 2023.
- [BFR24] Ryad Benadjila, Thibault Feneuil, and Matthieu Rivain. Mq on my mind: Post-quantum signatures from the non-structured multivariate quadratic problem. In *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, pages 468–485. IEEE, 2024.
- [BGS15] Sonia Belaïd, Vincent Grosso, and François-Xavier Standaert. Masking and leakage-resilient primitives: One, the other (s) or both? *Cryptography and Communications*, 7(1):163–184, 2015.
- [BHK⁺19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.
- [BN20] Carsten Baum and Ariel Nof. Concretely-efficient zero-knowledge arguments for arithmetic circuits and their application to lattice-based cryptography. In *IACR International Conference on Public-Key Cryptography*, pages 495–526. Springer, 2020.
- [CDG⁺17] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In *Proceedings of the 2017 acm sigsac conference on computer and communications security*, pages 1825–1842, 2017.
- [CDP17] Eleonora Cagli, Cécile Dumas, and Emmanuel Prouff. Convolutional neural networks with data augmentation against jitter-based countermeasures: Profiling attacks without pre-processing. In *International Conference on Cryptographic Hardware and Embedded Systems*, pages 45–68. Springer, 2017.
- [CJRR99] Suresh Chari, Charanjit S Jutla, Josyula R Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In *Annual International Cryptology Conference*, pages 398–412. Springer, 1999.
- [CRR02] Suresh Chari, Josyula R Rao, and Pankaj Rohatgi. Template attacks. In *International workshop on cryptographic hardware and embedded systems*, pages 13–28. Springer, 2002.
- [DLL⁺18] Léo Ducas, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: Digital signatures from module lattices. 2018.

- [FHK⁺18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-fourier lattice-based compact signatures over ntru. *Submission to the NIST's post-quantum cryptography standardization process*, 36(5):1–75, 2018.
- [FJR22] Thibault Feneuil, Antoine Joux, and Matthieu Rivain. Syndrome decoding in the head: Shorter signatures from zero-knowledge proofs. In *Annual International Cryptology Conference*, pages 541–572. Springer, 2022.
- [FR25] Thibault Feneuil and Matthieu Rivain. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *Journal of Cryptology*, 38(3):28, 2025.
- [FRWJ25] Thibault Feneuil, Matthieu Rivain, and Auguste Warmé-Janville. Masking-friendly post-quantum signatures in the threshold-computation-in-the-head framework. *Cryptology ePrint Archive*, 2025.
- [GAG⁺25] Julie Godard, Nicolas Aragon, Philippe Gaborit, Antoine Loiseau, and Julien Maillard. Single trace side-channel attack on the mpc-in-the-head framework. In *International Conference on Post-Quantum Cryptography*, pages 267–293. Springer, 2025.
- [GSE21] Tim Gellersen, Okan Seker, and Thomas Eisenbarth. Differential power analysis of the picnic signature scheme. In *International Conference on Post-Quantum Cryptography*, pages 177–194. Springer, 2021.
- [HOM06] Christoph Herbst, Elisabeth Oswald, and Stefan Mangard. An aes smart card implementation resistant to power analysis attacks. In *International conference on applied cryptography and network security*, pages 239–252. Springer, 2006.
- [IKOS09] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge proofs from secure multiparty computation. *SIAM Journal on Computing*, 39(3):1121–1152, 2009.
- [ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In *Advances in Cryptology-CRYPTO 2003: 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003. Proceedings 23*, pages 463–481. Springer, 2003.
- [KGB⁺18] Matthias J Kannwischer, Aymeric Genêt, Denis Butin, Juliane Krämer, and Johannes Buchmann. Differential power analysis of xmss and sphincs. In *International Workshop on Constructive Side-Channel Analysis and Secure Design*, pages 168–188. Springer, 2018.
- [KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Annual international cryptology conference*, pages 388–397. Springer, 1999.
- [Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of computation*, 48(177):203–209, 1987.
- [KZ20] Daniel Kales and Greg Zaverucha. An attack on some signature schemes constructed from five-pass identification schemes. In *International Conference on Cryptology and Network Security*, pages 3–22. Springer, 2020.
- [LBC17] Owen Lo, William J Buchanan, and Douglas Carson. Power analysis attacks on the aes-128 s-box using differential power analysis (dpa) and correlation power analysis (cpa). *Journal of cyber security technology*, 1(2):88–107, 2017.

- [MBB⁺25] C Aguilar Melchor, Slim Bettaieb, Loic Bidoux, Thibauld Feneuil, Philippe Gaborit, Nicolas Gama, Shay Gueron, James Howe, Andreas Hulsing, David Joseph, Antoine Joux, Mukul Kulkarni, Edoardo Persichetti, Tovahery H Randrianarisoa, Matthieu Rivain, and Yue Dongze. The syndrome decoding in the head (sd-in-the-head) signature scheme: Specification document (v2.0). *NIST Round 2 submission to the Additional Call for Signature Schemes*, 2025.
- [MFG⁺23] C Aguilar Melchor, Thibauld Feneuil, Nicolas Gama, Shay Gueron, James Howe, David Joseph, Antoine Joux, Edoardo Persichetti, Tovahery H Randrianarisoa, Matthieu Rivain, et al. The syndrome decoding in the head (sd-in-the-head) signature scheme. *NIST Round 1 submission to the Additional Call for Signature Schemes*, 2023.
- [Mil85] Victor S Miller. Use of elliptic curves in cryptography. In *Conference on the theory and application of cryptographic techniques*, pages 417–426. Springer, 1985.
- [NIS] NIST. Post-quantum cryptography standardization. <https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization>.
- [NIS22] NIST. Call for additional digital signature schemes for the post-quantum cryptography standardization process, 2022. Accessed: 2024-10-30.
- [NIS24] NIST. Ir 8528: Status report on the first round of the additional digital signature schemes for the nist post-quantum cryptography standardization process. Technical report, U.S. Department of Commerce, Washington, D.C., 2024.
- [PP19] Peter Pessl and Robert Primas. More practical single-trace attacks on the number theoretic transform. In *International Conference on Cryptology and Information Security in Latin America*, pages 130–149. Springer, 2019.
- [PS00] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of cryptology*, 13(3):361–396, 2000.
- [RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [SBWE20] Okan Seker, Sebastian Berndt, Luca Wilke, and Thomas Eisenbarth. Sni-in-the-head: Protecting mpc-in-the-head protocols against side-channel analysis. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1033–1049, 2020.